SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA11

COURSE OUTCOME COVERED

CO1: *Apply object-oriented concepts and software development life cycle models to design structured and modular software systems.* *(BL2, BL3)*

Assessment Tool Weightage: 30%

Dr Gayathri A

QUESTIONS: 6 **Total Marks:30**

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I ***D Varshini (`192320007)****(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Short Answer Test

1.Suppose a software system is developed for a **Smart Banking System** where customers can open accounts, transfer funds, and pay utility bills.

(a) Explain the following Object-Oriented Concepts

1. Class

A class is a blueprint or template used to create objects. It defines the attributes (data) and methods (functions) of an object.

Example:

BankAccount class contains attributes like accountNumber, balance, and methods such as deposit(), withdraw(), and transfer().

2. Object

An object is an instance of a class that contains actual values and can perform actions.

Example:

A customer's savings account with account number 123456789 is an object of the BankAccount class.

3. Message Passing

Message passing is the communication between objects by calling each other's methods.

Example:

When a customer transfers money, the Customer object sends a message to the BankAccount object to execute the transferFunds() method.

4. Encapsulation

Encapsulation is the process of combining data and methods into a single unit while restricting direct access to data.

Example:

The balance attribute is private and can only be modified through methods like deposit() or withdraw(), preventing unauthorized access.

5. Polymorphism

Polymorphism allows the same method to perform different actions depending on the object.

Example:

The calculateInterest() method works differently for SavingsAccount and CurrentAccount, although both use the same method name.

(b) Object Relationships

1. Association

Association is a relationship where two objects are connected but can exist independently.

Example:

A Customer is associated with a BankAccount. If the customer record is removed, the bank account may still exist in the system.

2. Aggregation

Aggregation is a "has-a" relationship where one object contains another object, but both can exist independently.

Example:

A Bank has many Customers. Even if the bank branch is closed, customer records may still exist in another branch.

3. Composition

Composition is a strong "part-of" relationship where the contained object cannot exist without the parent object.

Example:

A Transaction belongs to a specific BankAccount. If the account is permanently deleted, its transaction history is also removed.

(c) How These Concepts Improve Software Reusability and Maintainability

- Classes allow developers to reuse code by creating multiple objects from the same blueprint.
- Encapsulation protects important banking data such as account balance and PIN, improving security and reducing errors.
- Polymorphism allows new account types (e.g., Fixed Deposit or Loan Account) to be added without changing existing code.
- Association, Aggregation, and Composition organize relationships between banking objects, making the system easier to understand and modify.

- Message Passing enables objects to communicate efficiently without exposing internal details, reducing code dependency.
- Overall, these object-oriented concepts make the banking system modular, reusable, secure, scalable, and easy to maintain.

2.Consider developing a Smart Healthcare Management System.

(a) Explain the following Object-Oriented Design Principles

1. Abstraction

Abstraction means showing only the essential features of an object while hiding unnecessary implementation details.

Example:

In a Smart Healthcare Management System, a Doctor can access a patient's medical records through the system without knowing how the records are stored in the database.

2. Encapsulation

Encapsulation is the process of combining data and methods into a single unit while restricting direct access to the data.

Example:

The Patient class keeps details like medical history and personal information private. They can only be accessed or updated through authorized methods, ensuring data security.

3. Modularity

Modularity means dividing a software system into independent modules, each responsible for a specific task.

Example:

The healthcare system can have separate modules for:

- Patient Management
- Appointment Scheduling
- Billing
- Pharmacy
- Laboratory Reports

Each module can be developed and maintained independently.

(b) SOLID Principles

1. Liskov Substitution Principle (LSP)

The Liskov Substitution Principle states that a subclass should be able to replace its parent class without affecting the correctness of the program.

Example:

If Doctor is the parent class, then GeneralPhysician and Cardiologist should be usable wherever a Doctor object is expected without changing the system's behavior.

2. Interface Segregation Principle (ISP)

The Interface Segregation Principle states that a class should not be forced to implement methods it does not need. Instead, use smaller, specific interfaces.

Example:

Instead of one large interface called HealthcareServices, create separate interfaces such as:

- AppointmentService
- PrescriptionService
- BillingService

A receptionist implements only appointment-related services, while a pharmacist implements prescription-related services.

(c) How These Principles Improve Software Quality and Flexibility

- Abstraction simplifies the system by hiding complex implementation details from users.
- Encapsulation protects sensitive patient information and prevents unauthorized access.
- Modularity makes the system easier to develop, test, update, and maintain since each module works independently.
- Liskov Substitution Principle (LSP) ensures subclasses can replace parent classes without causing errors, improving code reliability.
- Interface Segregation Principle (ISP) avoids unnecessary methods in classes, making the code simpler and easier to maintain.
- Together, these principles make the Smart Healthcare Management System secure, scalable, reusable, flexible, and easy to maintain.

3.Suppose a software company is developing an **Online Food Delivery System**.

(a) Phases of the Software Development Life Cycle (SDLC)

1. Planning

The planning phase identifies the project goals, scope, budget, timeline, and required resources.
Example:
The company decides to develop an Online Food Delivery System with features like restaurant listing, food ordering, online payment, and order tracking.

2. Requirement Analysis

In this phase, developers gather and analyze the needs of users and stakeholders.
Example:
Customers need food search, order placement, and payment options, while restaurants need menu management and order tracking.

3. Design

The design phase creates the system architecture, database structure, user interface, and workflow.
Example:
Designing the customer app, restaurant dashboard, delivery partner interface, and database for orders and payments.

4. Development

Developers write the program code based on the design.
Example:
Implementing features such as user registration, restaurant search, cart, payment gateway, and live order tracking.

5. Testing

The software is tested to identify and fix errors before release.
Example:
Testing login, order placement, payment processing, and delivery tracking to ensure they work correctly.

6. Maintenance

After deployment, the software is updated to fix bugs, improve performance, and add new features.

Example:

Adding discount coupons, improving delivery tracking, and fixing issues reported by users.

(b) Purpose of Each SDLC Phase

SDLC Phase	Purpose
Planning	Defines project objectives, budget, timeline, and feasibility.
Requirement Analysis	Collects and documents user and business requirements.
Design	Creates the system architecture, database, and user interface.
Development	Converts the design into working software through coding.
Testing	Detects and fixes bugs to ensure software quality.
Maintenance	Supports the software after release by fixing issues and adding enhancements.

(c) How SDLC Contributes to Delivering Reliable Software

- Planning ensures the project has clear goals and proper resource allocation.
- Requirement Analysis helps developers build software that meets user needs.
- Design provides a well-structured architecture, making the system efficient and scalable.
- Development implements features according to the approved design.
- Testing identifies and removes defects, improving software quality and reliability.
- Maintenance keeps the system updated, secure, and compatible with changing user requirements.
- Overall, SDLC ensures the Online Food Delivery System is high-quality, reliable, secure, maintainable, and delivered on time.

4.Consider a project for developing a **Smart Agriculture Monitoring System**, where customer requirements change regularly.

(a) Explain the following SDLC Models

1. Prototype Model

The Prototype Model creates a working prototype of the software to gather user feedback before developing the final system.

Example:
A basic prototype of the agriculture monitoring system is developed with features like crop monitoring and weather updates for farmers to review.

2. Spiral Model

The Spiral Model combines iterative development with risk analysis. Each cycle includes planning, risk assessment, development, and testing.

Example:
The system is developed in stages by first implementing sensor monitoring, then weather prediction, while analyzing risks such as sensor failures.

3. Agile Model

The Agile Model develops software in small iterations called sprints. Customer feedback is collected after each sprint to improve the system.

Example:
Features like soil moisture monitoring, irrigation control, and pest detection are released in separate sprints.

4. DevOps Model

The DevOps Model integrates development and operations by automating testing, deployment, and monitoring for faster software delivery.

Example:

New features and bug fixes are automatically tested and deployed without interrupting the monitoring system.

(b) Compare Prototype Model and Agile Model

Prototype Model	Agile Model
Builds a prototype before final development.	Develops software in multiple short iterations (sprints).
Used when requirements are unclear.	Used when requirements change frequently.
Feedback is mainly collected during prototype evaluation.	Feedback is collected after every sprint.
Final product is developed after prototype approval.	Software is continuously improved throughout development.
Less flexible after development begins.	Highly flexible and adaptable to changes.

(c) Most Suitable SDLC Model with Justification

The Agile Model is the most suitable for the Smart Agriculture Monitoring System because customer requirements change regularly.

Justification:

- Easily accommodates changing requirements.
- Delivers new features quickly through short sprints.
- Continuous customer feedback improves the system.
- Bugs are fixed early through frequent testing.
- Supports continuous improvement and faster delivery.

Conclusion:

The Agile Model is the best choice because it provides flexibility, continuous feedback, faster development, and easy adaptation to changing customer needs

5.Suppose a software analyst is designing a **Smart Transport Management System**.

(a) Importance of UML in Object-Oriented Software Development

Unified Modeling Language (UML) is a standard graphical language used to visualize, design, and document object-oriented software systems.

Importance:

- Provides a visual representation of the system.
- Helps developers understand system structure and behavior.
- Improves communication among developers, clients, and stakeholders.
- Detects design issues before coding.
- Makes software easier to maintain and modify.

(b) UML Diagrams

1. State Diagram

A State Diagram shows the different states of an object and the transitions between them.

Example:

A bus changes states as:

Available → Assigned → In Transit → Maintenance → Available.

2. Component Diagram

A Component Diagram shows the major software components and their relationships.

Example:

Components include:

- Passenger Module
- Vehicle Module
- Ticket Booking Module
- Payment Module
- Database

3. Deployment Diagram

A Deployment Diagram shows how software components are deployed on hardware devices.

Example:

The mobile app connects to the application server and database server through the internet.

4. Package Diagram

A Package Diagram groups related classes into packages for better organization.

Example:

Packages include:

- User Management
- Vehicle Management
- Booking Management
- Payment Management
- Notification Management

(c) How UML Models Improve Communication Among Stakeholders

- UML provides a common visual language for developers, testers, clients, and managers.
- It helps stakeholders understand system design before coding begins.
- Visual diagrams reduce misunderstandings and clarify requirements.
- Makes discussions and future modifications easier.
- Improves documentation for maintenance and future enhancements.

Conclusion:

UML improves communication by providing clear visual models, helping all stakeholders understand the system and reducing development errors.

6.Consider developing a **Smart Inventory Management System**.

(a) Explain the Concept of Modular Software Design

Modular software design is the process of dividing a software system into small, independent modules, where each module performs a specific function.

Example:

Modules in a Smart Inventory Management System include:

- Product Management
- Inventory Tracking
- Supplier Management
- Order Management
- Billing and Reports

Each module can be developed and maintained independently.

(b) Compare Procedural Programming and Object-Oriented Programming

Procedural Programming	Object-Oriented Programming (OOP)
Focuses on functions or procedures.	Focuses on classes and objects.
Data and functions are separate.	Data and methods are combined (encapsulation).
Uses a top-down approach.	Uses a bottom-up approach.
Less secure because data is easily accessible.	More secure due to data hiding.
Limited code reusability.	High code reusability using inheritance and polymorphism.
Difficult to maintain large systems.	Easier to maintain and extend large applications.

(c) How Modularity Improves Software Scalability, Maintainability, and Reusability

- **Scalability:** New modules, such as barcode scanning or warehouse management, can be added without affecting existing modules.
- **Maintainability:** Bugs and updates can be handled in one module without changing the entire system.
- **Reusability:** Existing modules, such as billing or authentication, can be reused in other projects.
- **Independent Development:** Different teams can work on separate modules simultaneously.
- **Easy Testing:** Each module can be tested independently, making debugging easier.

Conclusion:

Modularity makes the Smart Inventory Management System scalable, maintainable, reusable, and easier to develop, resulting in better software quality and lower maintenance effort.

RUBRICS FOR CALCULATION

SHORT ANSWER TEST

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	20%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps